```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from tensorflow.keras import models,layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Flatten
from sklearn.metrics import classification_report,confusion_matrix
```

## loading dataset

```
#main_dir = '/content/drive/MyDrive/'
main_dir = '../input/plant-seedlings-classification'
train_dir = os.path.join(main_dir,"train")
test_dir = os.path.join(main_dir,"test")


labels = sorted(os.listdir(train_dir))
print(labels)
```

```
['Black-grass', 'Charlock', 'Cleavers', 'Common Chickweed', 'Common
wheat', 'Fat Hen', 'Loose Silky-bent', 'Maize', 'Scentless Mayweed',
'Shepherds Purse', 'Small-flowered Cranesbill', 'Sugar beet']
```

```
num_classes = len(labels)
num_classes
```

```
12
```

```
classes = list(labels)

classes
```

```
['Black-grass',
 'Charlock',
 'Cleavers',
 'Common Chickweed',
 'Common wheat',
 'Fat Hen',
 'Loose Silky-bent',
 'Maize',
 'Scentless Mayweed',
 'Shepherds Purse',
 'Small-flowered Cranesbill',
 'Sugar beet']
```

## preprocessing

```
batch_size = 32
img_height = 224
img_width = 224
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale = 1./255.,
    rotation_range = 40,
    width_shift_range = 0.2,
    height_shift_range = 0.2,
    shear_range = 0.2,
    zoom_range = 0.2,
    validation_split=0.20,
    horizontal_flip = True
    )

train_generator = train_datagen.flow_from_directory(train_dir,
                                                    batch_size = 20,
                                                    subset =
'training',
                                                    target_size =
(img_height, img_width))



Found 3803 images belonging to 12 classes.

validation_generator = train_datagen.flow_from_directory(train_dir,
                                                         batch_size =
20,

subset='validation',
                                                         target_size =
(img_height, img_width))

Found 947 images belonging to 12 classes.
```

## VGG

```
from keras.applications.vgg16 import
preprocess_input,decode_predictions,VGG16
base_model = VGG16( input_shape = (img_height,img_width, 3),
include_top = False,
weights = 'imagenet')

Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58892288/58889256 [==============================] - 0s 0us/step
58900480/58889256 [==============================] - 0s 0us/step
```

18 layers

```
# for i, layer in enumerate(base_model.layers):
#     print(i, layer.name, layer.trainable)

for layer in base_model.layers[:15]:
    layer.trainable = False

# for i, layer in enumerate(base_model.layers):
#     print(i, layer.name, layer.trainable)

x = base_model.output
x = layers.Flatten()(x)
x = Dense(512, activation='relu')(x)
x = layers.Dropout(0.5)(x)
x = Dense(256, activation='relu')(x)
x = Dense(12, activation='softmax')(x)
vgg_model = models.Model(inputs=base_model.input, outputs=x)
```

```
from keras.callbacks import ReduceLROnPlateau, ModelCheckpoint,
EarlyStopping
lr_reduce = ReduceLROnPlateau(monitor='val_accuracy', factor=0.6,
patience=5, verbose=1, mode='max', min_lr=5e-5)

checkpoint = ModelCheckpoint('vgg.h5', monitor= 'val_accuracy', mode=
'max', save_best_only = True, verbose= 1)

early_stop=EarlyStopping(monitor='val_loss',mode='min',verbose=1,patie
nce=5)
callbacklist = [lr_reduce,checkpoint,early_stop]
```

```
from tensorflow.keras import layers, models, Model, optimizers
learning_rate= 5e-5
vgg_model.compile(loss="categorical_crossentropy",
optimizer=optimizers.Adam(lr=learning_rate), metrics=["accuracy"])

vgghistory = vgg_model.fit_generator(
    train_generator,
    validation_data=validation_generator,
    epochs=10,
    shuffle=True,
    callbacks =callbacklist,
    verbose=1)
```

```
/opt/conda/lib/python3.7/site-packages/keras/optimizer_v2/
optimizer_v2.py:356: UserWarning: The `lr` argument is deprecated, use
`learning_rate` instead.
  "The `lr` argument is deprecated, use `learning_rate` instead.")

Epoch 1/10
191/191 [==============================] - 111s 568ms/step - loss:
2.0977 - accuracy: 0.2727 - val_loss: 1.6534 - val_accuracy: 0.4372
```

```
Epoch 00001: val_accuracy improved from -inf to 0.43717, saving model
to vgg.h5
Epoch 2/10
191/191 [==============================] - 107s 560ms/step - loss:
1.4668 - accuracy: 0.4978 - val_loss: 1.0955 - val_accuracy: 0.6304

Epoch 00002: val_accuracy improved from 0.43717 to 0.63041, saving
model to vgg.h5
Epoch 3/10
191/191 [==============================] - 107s 561ms/step - loss:
1.0778 - accuracy: 0.6211 - val_loss: 0.8961 - val_accuracy: 0.6927

Epoch 00003: val_accuracy improved from 0.63041 to 0.69271, saving
model to vgg.h5
Epoch 4/10
191/191 [==============================] - 109s 570ms/step - loss:
0.8565 - accuracy: 0.7034 - val_loss: 0.7773 - val_accuracy: 0.7434

Epoch 00004: val_accuracy improved from 0.69271 to 0.74340, saving
model to vgg.h5
Epoch 5/10
191/191 [==============================] - 107s 558ms/step - loss:
0.7708 - accuracy: 0.7310 - val_loss: 0.6262 - val_accuracy: 0.7825

Epoch 00005: val_accuracy improved from 0.74340 to 0.78247, saving
model to vgg.h5
Epoch 6/10
191/191 [==============================] - 106s 557ms/step - loss:
0.6533 - accuracy: 0.7725 - val_loss: 0.5784 - val_accuracy: 0.8068

Epoch 00006: val_accuracy improved from 0.78247 to 0.80676, saving
model to vgg.h5
Epoch 7/10
191/191 [==============================] - 106s 555ms/step - loss:
0.6088 - accuracy: 0.7910 - val_loss: 0.5348 - val_accuracy: 0.8215

Epoch 00007: val_accuracy improved from 0.80676 to 0.82154, saving
model to vgg.h5
Epoch 8/10
191/191 [==============================] - 105s 552ms/step - loss:
0.5597 - accuracy: 0.8057 - val_loss: 0.5698 - val_accuracy: 0.8036

Epoch 00008: val_accuracy did not improve from 0.82154
Epoch 9/10
191/191 [==============================] - 106s 554ms/step - loss:
0.5049 - accuracy: 0.8257 - val_loss: 0.5558 - val_accuracy: 0.8184

Epoch 00009: val_accuracy did not improve from 0.82154
```

```
Epoch 10/10
191/191 [==============================] - 106s 553ms/step - loss:
0.4959 - accuracy: 0.8220 - val_loss: 0.5744 - val_accuracy: 0.8237

Epoch 00010: val_accuracy improved from 0.82154 to 0.82365, saving
model to vgg.h5
```

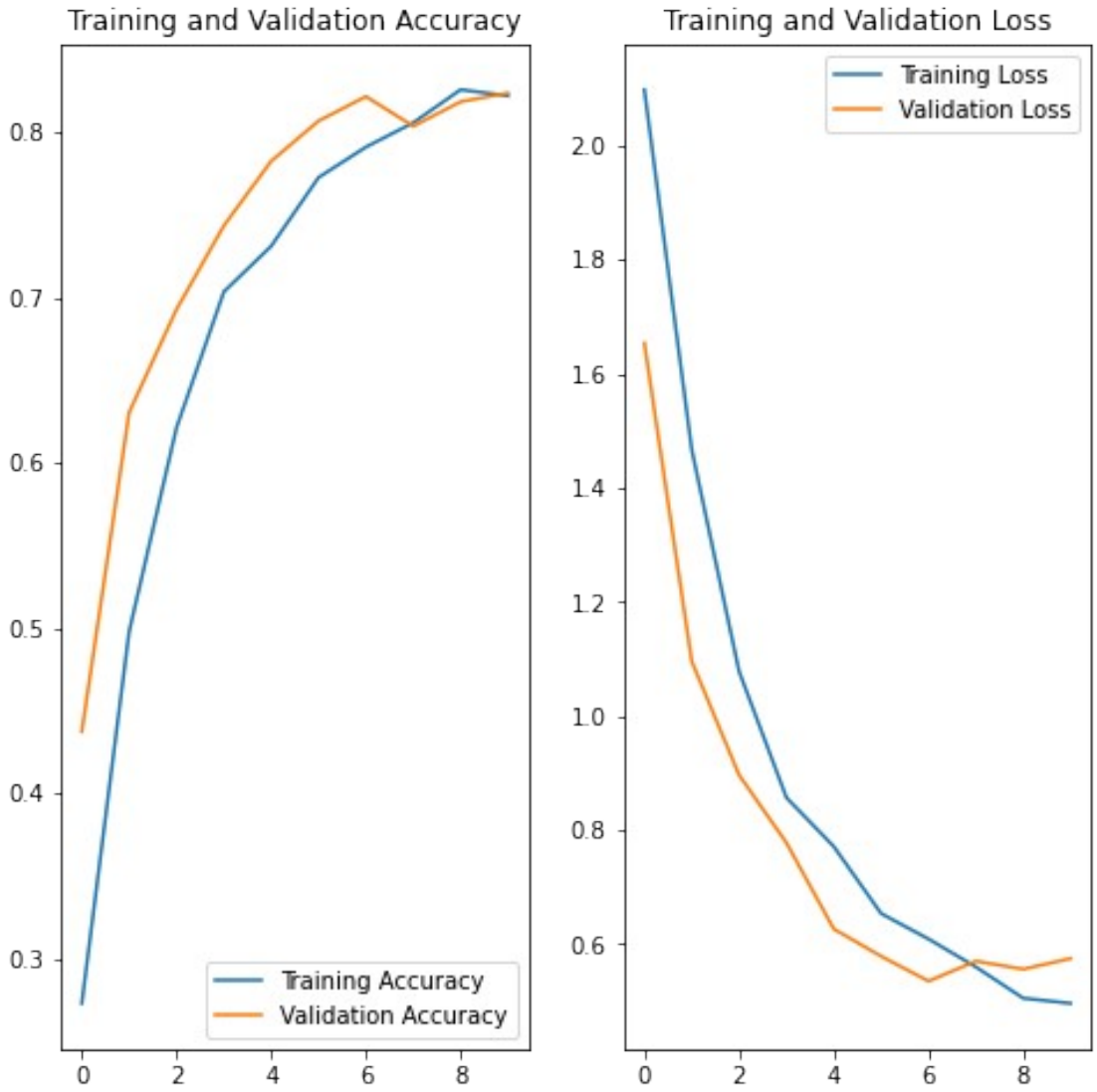
```
acc = vgghistory.history['accuracy']
val_acc = vgghistory.history['val_accuracy']

loss = vgghistory.history['loss']
val_loss = vgghistory.history['val_loss']

epochs_range = range(10)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```




```
#vgg_model.save('vgg.h5')
```

```
y_pred1 = vgg_model.predict(validation_generator)
y_pred1 = np.argmax(y_pred1, axis=1)
y_pred1.sort()

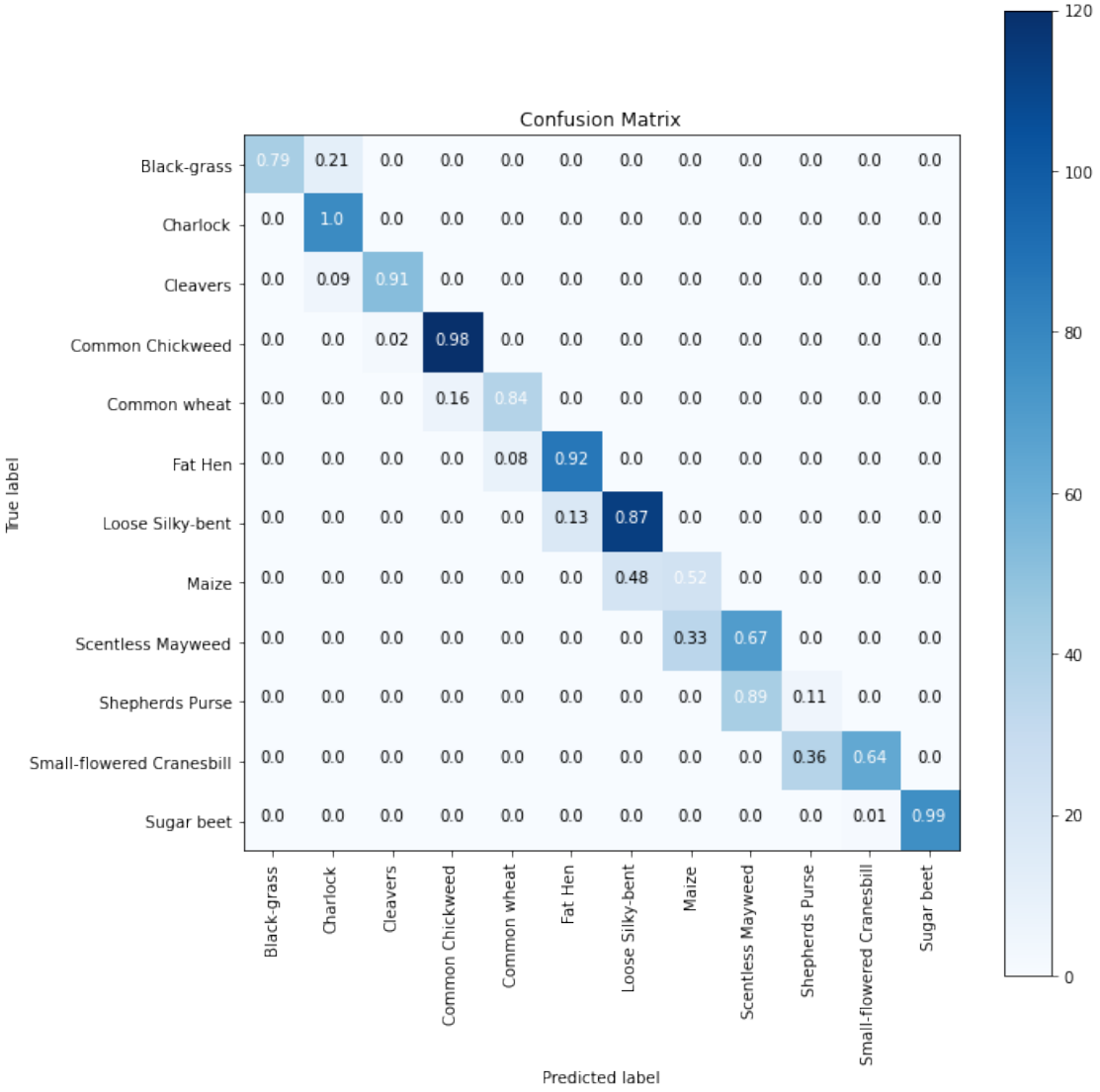
cm1 = confusion_matrix(validation_generator.classes, y_pred1)

import itertools
def plot_confusion_matrix(cm, classes, normalize=True,
title='Confusion matrix', cmap=plt.cm.Blues):
    plt.figure(figsize=(10,10))
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=90)
```

```
    plt.yticks(tick_marks, classes)
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        cm = np.around(cm, decimals=2)
        cm[np.isnan(cm)] = 0.0
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]),
range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")
    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

plot_confusion_matrix(cm1, classes, title='Confusion Matrix')
```

```
Normalized confusion matrix
```




```
print(classification_report(validation_generator.classes,
y_pred1,target_names = classes))
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Black-grass | 1.00 | 0.79 | 0.88 | 52 |
| Charlock | 0.83 | 1.00 | 0.91 | 78 |
| Cleavers | 0.96 | 0.91 | 0.94 | 57 |
| Common Chickweed | 0.94 | 0.98 | 0.96 | 122 |
| Common wheat | 0.82 | 0.84 | 0.83 | 44 |
| Fat Hen | 0.84 | 0.92 | 0.87 | 95 |
| Loose Silky-bent | 0.84 | 0.87 | 0.86 | 130 |
| Maize | 0.40 | 0.52 | 0.46 | 44 |
| Scentless Mayweed | 0.63 | 0.67 | 0.65 | 103 |
| Shepherds Purse | 0.12 | 0.11 | 0.11 | 46 |
| Small-flowered Cranesbill | 0.98 | 0.64 | 0.77 | 99 |
| Sugar beet | 1.00 | 0.99 | 0.99 | 77 |

| | | | | |
|---|---|---|---|---|
| accuracy | | | 0.81 | 947 |
| macro avg | 0.78 | 0.77 | 0.77 | 947 |
| weighted avg | 0.82 | 0.81 | 0.81 | 947 |

## RESNET

```
from tensorflow.keras.applications.resnet50 import ResNet50
base_model1 = ResNet50( input_shape = (img_height,img_width,
3),include_top = False, weights = 'imagenet')
```

```
2022-03-08 14:30:16.815498: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:16.907008: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:16.907708: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:16.908768: I
tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow
binary is optimized with oneAPI Deep Neural Network Library (oneDNN)
to use the following CPU instructions in performance-critical
operations:  AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the
appropriate compiler flags.
2022-03-08 14:30:16.909731: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:16.910510: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:16.911133: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:18.647972: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:18.648749: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
```

```
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:18.649440: I
tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful
NUMA node read from SysFS had negative value (-1), but there must be
at least one NUMA node, so returning NUMA node zero
2022-03-08 14:30:18.650030: I
tensorflow/core/common_runtime/gpu/gpu_device.cc:1510] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 15403 MB
memory:  -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id:
0000:00:04.0, compute capability: 6.0

Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/resnet/
resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94773248/94765736 [==============================] - 2s 0us/step
94781440/94765736 [==============================] - 2s 0us/step
```

```
174 layers
# for i, layer in enumerate(base_model1.layers):
#     print(i, layer.name, layer.trainable)


for layer in base_model1.layers[:15]:
    layer.trainable = False

x1 = base_model1.output
x1 = layers.Flatten()(x1)
x1 = Dense(512, activation='relu')(x1)
x1 = layers.Dropout(0.5)(x1)
x1 = Dense(256, activation='relu')(x1)

x1 = Dense(12, activation='softmax')(x1)
resnet_model = models.Model(inputs=base_model1.input, outputs=x1)

from keras.callbacks import ReduceLROnPlateau, ModelCheckpoint,
EarlyStopping
lr_reduce = ReduceLROnPlateau(monitor='val_accuracy', factor=0.05,
patience=5, verbose=1, mode='max', min_lr=0.000002)

checkpoint = ModelCheckpoint('resnet.h5', monitor= 'val_accuracy',
mode= 'max', save_best_only = True, verbose= 1)

early_stop=EarlyStopping(monitor='val_loss',mode='min',verbose=1,patie
nce=5)
callbacklist = [lr_reduce,checkpoint,early_stop]

from tensorflow.keras import layers, models, Model, optimizers
learning_rate= 5e-5
resnet_model.compile(
    loss="categorical_crossentropy",
```

```
    optimizer=optimizers.Adam(learning_rate=learning_rate),
metrics=["accuracy"])

resnethistory = resnet_model.fit_generator(
    train_generator,
    validation_data=validation_generator,
    epochs=10,
    shuffle=True,
    callbacks = callbacklist,
    verbose=1)
```

```
/opt/conda/lib/python3.7/site-packages/keras/engine/training.py:1972:
UserWarning: `Model.fit_generator` is deprecated and will be removed
in a future version. Please use `Model.fit`, which supports
generators.
  warnings.warn('`Model.fit_generator` is deprecated and '
2022-03-08 14:30:27.735287: I
tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:185] None of
the MLIR Optimization Passes are enabled (registered 2)

Epoch 1/10

2022-03-08 14:30:33.586734: I
tensorflow/stream_executor/cuda/cuda_dnn.cc:369] Loaded cuDNN version
8005

191/191 [==============================] - 165s 808ms/step - loss:
2.0792 - accuracy: 0.3879 - val_loss: 5.0151 - val_accuracy: 0.1373

Epoch 00001: val_accuracy improved from -inf to 0.13728, saving model
to resnet.h5

/opt/conda/lib/python3.7/site-packages/keras/utils/
generic_utils.py:497: CustomMaskWarning: Custom mask layers require a
config and must override get_config. When loading, the custom mask
layer must be passed to the custom_objects argument.
  category=CustomMaskWarning)

Epoch 2/10
191/191 [==============================] - 111s 584ms/step - loss:
0.8313 - accuracy: 0.7276 - val_loss: 3.6296 - val_accuracy: 0.1373

Epoch 00002: val_accuracy did not improve from 0.13728
Epoch 3/10
191/191 [==============================] - 110s 577ms/step - loss:
0.5915 - accuracy: 0.8025 - val_loss: 4.6474 - val_accuracy: 0.1373

Epoch 00003: val_accuracy did not improve from 0.13728
Epoch 4/10
191/191 [==============================] - 110s 575ms/step - loss:
0.4723 - accuracy: 0.8449 - val_loss: 5.6623 - val_accuracy: 0.0876
```

```
Epoch 00004: val_accuracy did not improve from 0.13728
Epoch 5/10
191/191 [==============================] - 131s 686ms/step - loss:
0.3857 - accuracy: 0.8669 - val_loss: 3.4809 - val_accuracy: 0.2735

Epoch 00005: val_accuracy improved from 0.13728 to 0.27350, saving
model to resnet.h5
Epoch 6/10
191/191 [==============================] - 111s 583ms/step - loss:
0.3386 - accuracy: 0.8896 - val_loss: 0.7764 - val_accuracy: 0.7835

Epoch 00006: val_accuracy improved from 0.27350 to 0.78353, saving
model to resnet.h5
Epoch 7/10
191/191 [==============================] - 111s 580ms/step - loss:
0.2901 - accuracy: 0.8961 - val_loss: 0.3172 - val_accuracy: 0.8986

Epoch 00007: val_accuracy improved from 0.78353 to 0.89863, saving
model to resnet.h5
Epoch 8/10
191/191 [==============================] - 111s 578ms/step - loss:
0.2728 - accuracy: 0.9032 - val_loss: 0.2549 - val_accuracy: 0.9187

Epoch 00008: val_accuracy improved from 0.89863 to 0.91869, saving
model to resnet.h5
Epoch 9/10
191/191 [==============================] - 111s 580ms/step - loss:
0.2476 - accuracy: 0.9098 - val_loss: 0.3141 - val_accuracy: 0.9208

Epoch 00009: val_accuracy improved from 0.91869 to 0.92080, saving
model to resnet.h5
Epoch 10/10
191/191 [==============================] - 111s 580ms/step - loss:
0.2352 - accuracy: 0.9209 - val_loss: 0.3165 - val_accuracy: 0.9081

Epoch 00010: val_accuracy did not improve from 0.92080
```

```
acc = resnethistory.history['accuracy']
val_acc = resnethistory.history['val_accuracy']

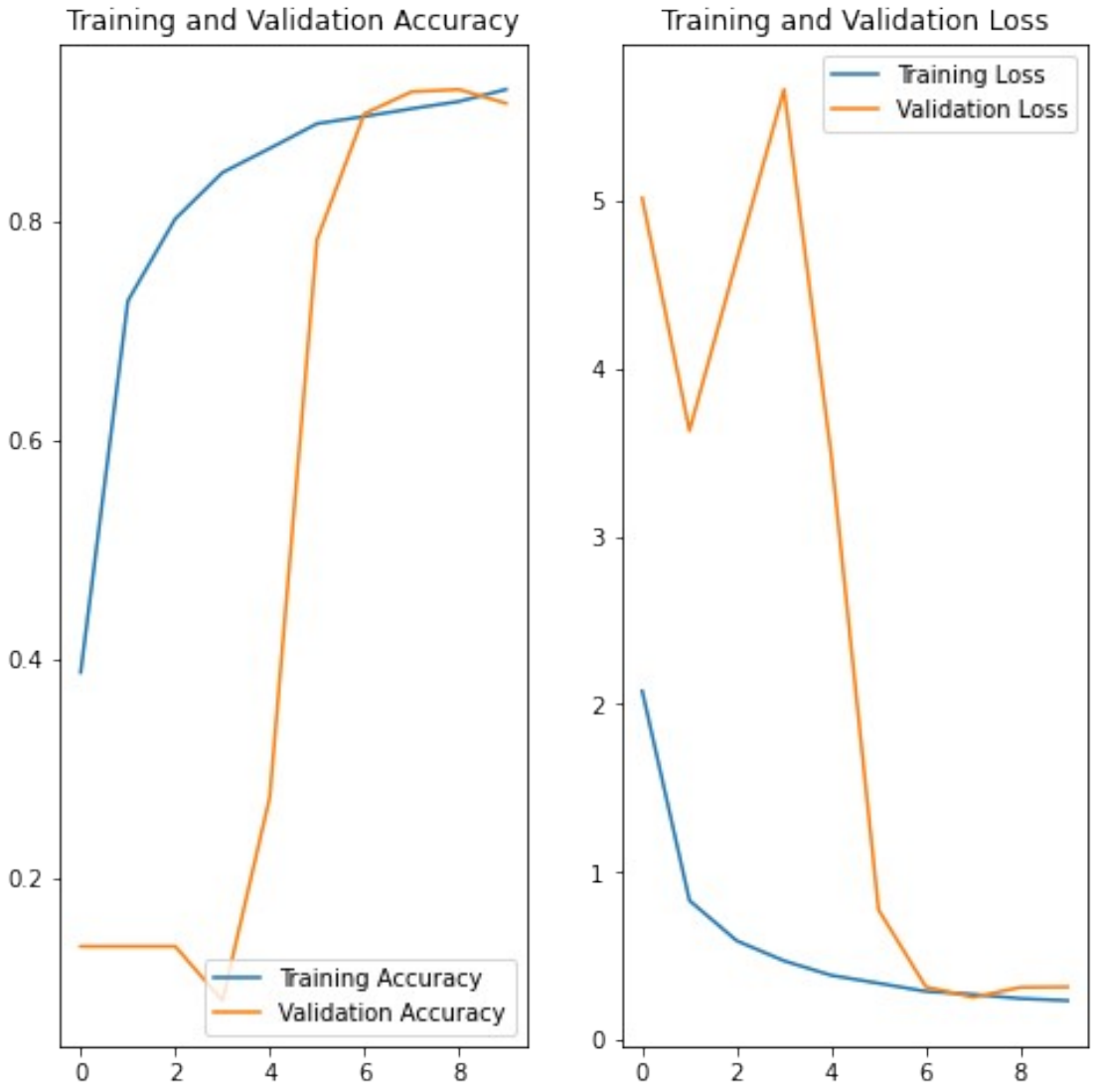
loss = resnethistory.history['loss']
val_loss = resnethistory.history['val_loss']

epochs_range = range(10)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
```

```
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

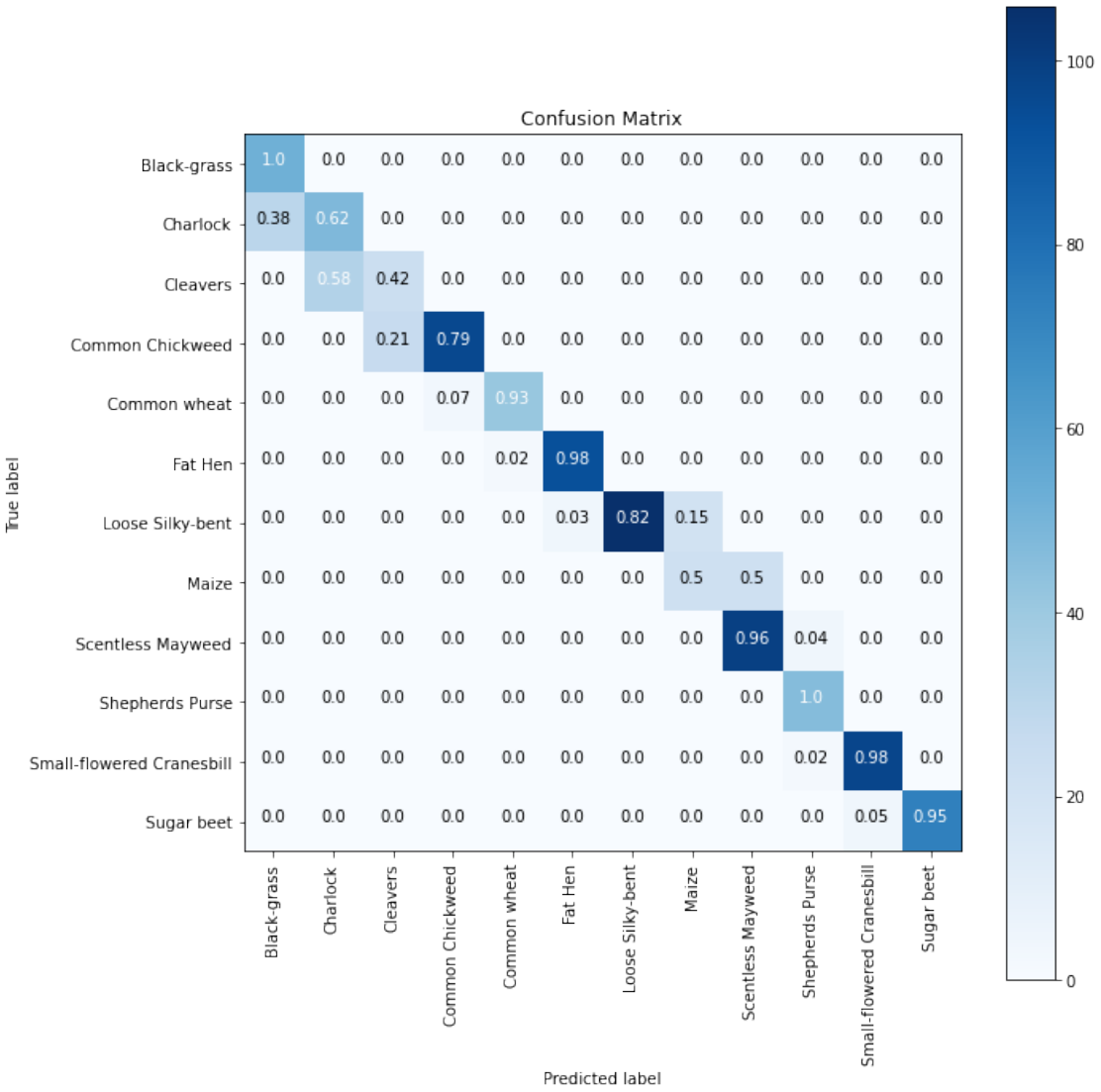



```
y_pred2 = resnet_model.predict(validation_generator)
y_pred2 = np.argmax(y_pred2, axis=1)
y_pred2.sort()

cm2 = confusion_matrix(validation_generator.classes, y_pred2)
```

```
plot_confusion_matrix(cm2, classes, title='Confusion Matrix')
```

```
Normalized confusion matrix
```




```
print(classification_report(validation_generator.classes,
y_pred2,target_names = classes))
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Black-grass | 0.63 | 1.00 | 0.78 | 52 |
| Charlock | 0.59 | 0.62 | 0.60 | 78 |
| Cleavers | 0.48 | 0.42 | 0.45 | 57 |
| Common Chickweed | 0.97 | 0.79 | 0.87 | 122 |
| Common wheat | 0.95 | 0.93 | 0.94 | 44 |
| Fat Hen | 0.96 | 0.98 | 0.97 | 95 |
| Loose Silky-bent | 1.00 | 0.82 | 0.90 | 130 |
| Maize | 0.52 | 0.50 | 0.51 | 44 |

```
        Scentless Mayweed       0.82      0.96      0.88       103
          Shepherds Purse       0.88      1.00      0.94        46
Small-flowered Cranesbill       0.96      0.98      0.97        99
               Sugar beet       1.00      0.95      0.97        77

                 accuracy                           0.84       947
                macro avg       0.81      0.83      0.82       947
             weighted avg       0.85      0.84      0.84       947
```

```
#resnet_model.save('resnet.h5')
```

## Inception

```
from tensorflow.keras.applications.inception_v3 import InceptionV3
base_model2 = InceptionV3(input_shape = (img_height,img_width, 3),
include_top = False, weights = 'imagenet')
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/inception_v3/
inception_v3_weights_tf_dim_ordering_tf_kernels_notop.h5
87916544/87910968 [==============================] - 0s 0us/step
87924736/87910968 [==============================] - 0s 0us/step
```

*310 layers*

```
# for i, layer in enumerate(base_model2.layers):
#     print(i, layer.name, layer.trainable)
```

```
for layer in base_model2.layers[:15]:
    layer.trainable = False
```

```
# for i, layer in enumerate(base_model2.layers):
#     print(i, layer.name, layer.trainable)
#310 layers
```

```
x2 = base_model2.output
x2 = layers.Flatten()(x2)
x2 = Dense(512, activation='relu')(x2)
x2 = layers.Dropout(0.5)(x2)
x2 = Dense(256, activation='relu')(x2)
x2 = Dense(12, activation='softmax')(x2)
inc_model = models.Model(inputs=base_model2.input, outputs=x2)
```

```
from keras.callbacks import ReduceLROnPlateau, ModelCheckpoint,
EarlyStopping
lr_reduce = ReduceLROnPlateau(monitor='val_accuracy', factor=0.6,
patience=5, verbose=1, mode='max', min_lr=5e-5)
```

```
checkpoint = ModelCheckpoint('./inc.h5', monitor= 'val_accuracy',
mode= 'max', save_best_only = True, verbose= 1)
early_stop=EarlyStopping(monitor='val_loss',mode='min',verbose=1,patie
```

```
nce=5)
callbacklist = [lr_reduce,early_stop,checkpoint]

from tensorflow.keras import layers, models, Model, optimizers
learning_rate= 5e-5
inc_model.compile(loss="categorical_crossentropy",
optimizer=optimizers.Adam(learning_rate=learning_rate),
metrics=["accuracy"])


inchistory = inc_model.fit_generator(
    train_generator,
    validation_data=validation_generator,
    epochs=10,
    shuffle=True,
    callbacks = callbacklist,
    verbose=1)
```

```
/opt/conda/lib/python3.7/site-packages/keras/engine/training.py:1972:
UserWarning: `Model.fit_generator` is deprecated and will be removed
in a future version. Please use `Model.fit`, which supports
generators.
  warnings.warn('`Model.fit_generator` is deprecated and '

Epoch 1/10
191/191 [==============================] - 116s 573ms/step - loss:
2.1244 - accuracy: 0.2948 - val_loss: 1.1717 - val_accuracy: 0.6272

Epoch 00001: val_accuracy improved from -inf to 0.62724, saving model
to ./inc.h5
Epoch 2/10
191/191 [==============================] - 110s 577ms/step - loss:
1.0579 - accuracy: 0.6487 - val_loss: 0.5493 - val_accuracy: 0.8194

Epoch 00002: val_accuracy improved from 0.62724 to 0.81943, saving
model to ./inc.h5
Epoch 3/10
191/191 [==============================] - 129s 675ms/step - loss:
0.6441 - accuracy: 0.7852 - val_loss: 0.3779 - val_accuracy: 0.8775

Epoch 00003: val_accuracy improved from 0.81943 to 0.87751, saving
model to ./inc.h5
Epoch 4/10
191/191 [==============================] - 107s 562ms/step - loss:
0.4901 - accuracy: 0.8393 - val_loss: 0.3813 - val_accuracy: 0.8817

Epoch 00004: val_accuracy improved from 0.87751 to 0.88173, saving
model to ./inc.h5
Epoch 5/10
191/191 [==============================] - 108s 566ms/step - loss:
0.4225 - accuracy: 0.8604 - val_loss: 0.2821 - val_accuracy: 0.9071
```

```
Epoch 00005: val_accuracy improved from 0.88173 to 0.90707, saving
model to ./inc.h5
Epoch 6/10
191/191 [==============================] - 108s 567ms/step - loss:
0.3821 - accuracy: 0.8725 - val_loss: 0.3111 - val_accuracy: 0.8986

Epoch 00006: val_accuracy did not improve from 0.90707
Epoch 7/10
191/191 [==============================] - 108s 568ms/step - loss:
0.3501 - accuracy: 0.8840 - val_loss: 0.3055 - val_accuracy: 0.9124

Epoch 00007: val_accuracy improved from 0.90707 to 0.91235, saving
model to ./inc.h5
Epoch 8/10
191/191 [==============================] - 108s 566ms/step - loss:
0.3147 - accuracy: 0.8948 - val_loss: 0.2847 - val_accuracy: 0.9018

Epoch 00008: val_accuracy did not improve from 0.91235
Epoch 9/10
191/191 [==============================] - 109s 568ms/step - loss:
0.2652 - accuracy: 0.9111 - val_loss: 0.2846 - val_accuracy: 0.9250

Epoch 00009: val_accuracy improved from 0.91235 to 0.92503, saving
model to ./inc.h5
Epoch 10/10
191/191 [==============================] - 109s 568ms/step - loss:
0.2360 - accuracy: 0.9138 - val_loss: 0.2352 - val_accuracy: 0.9282

Epoch 00010: val_accuracy improved from 0.92503 to 0.92819, saving
model to ./inc.h5
```

```
acc = inchistory.history['accuracy']
val_acc = inchistory.history['val_accuracy']

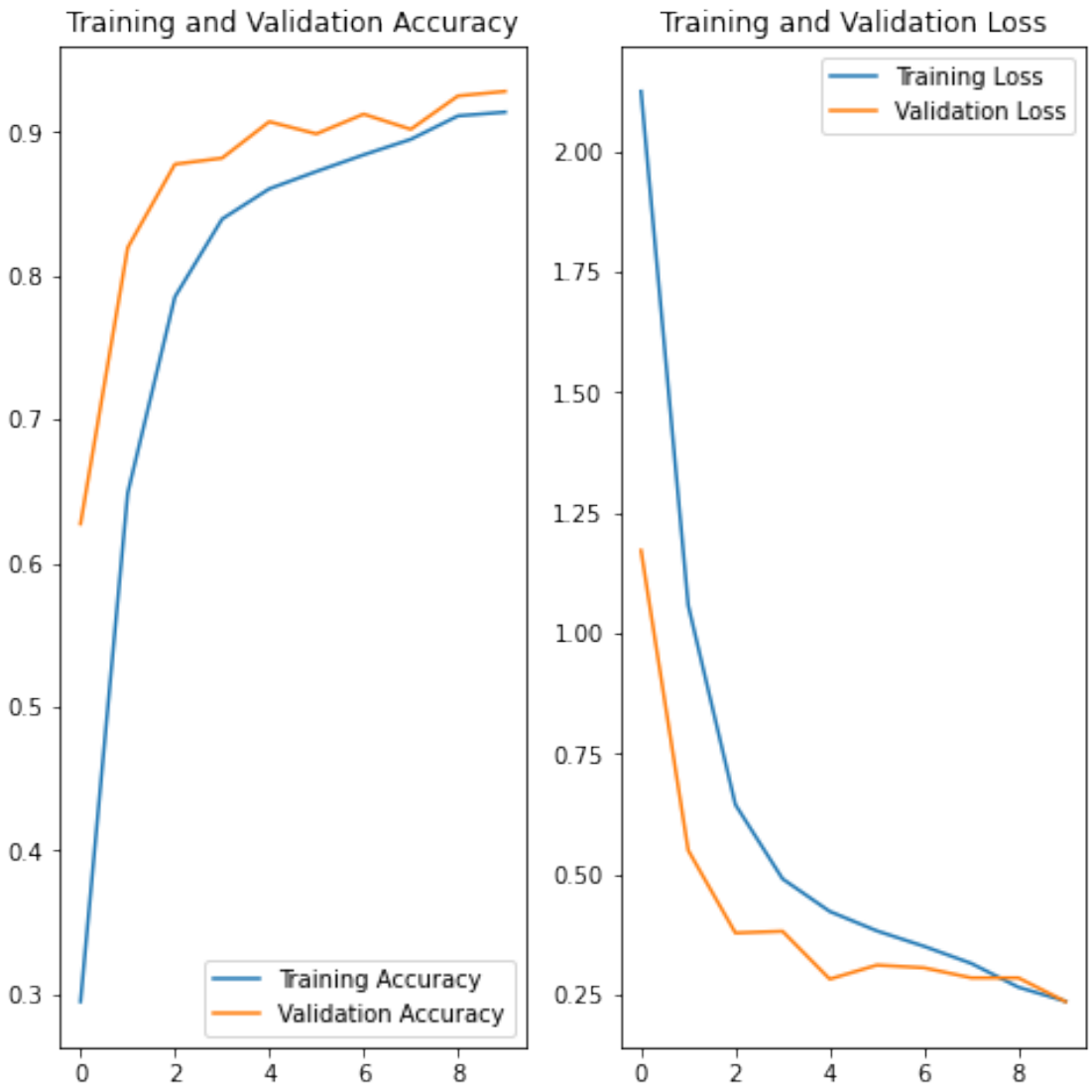
loss = inchistory.history['loss']
val_loss = inchistory.history['val_loss']

epochs_range = range(10)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
```

```
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

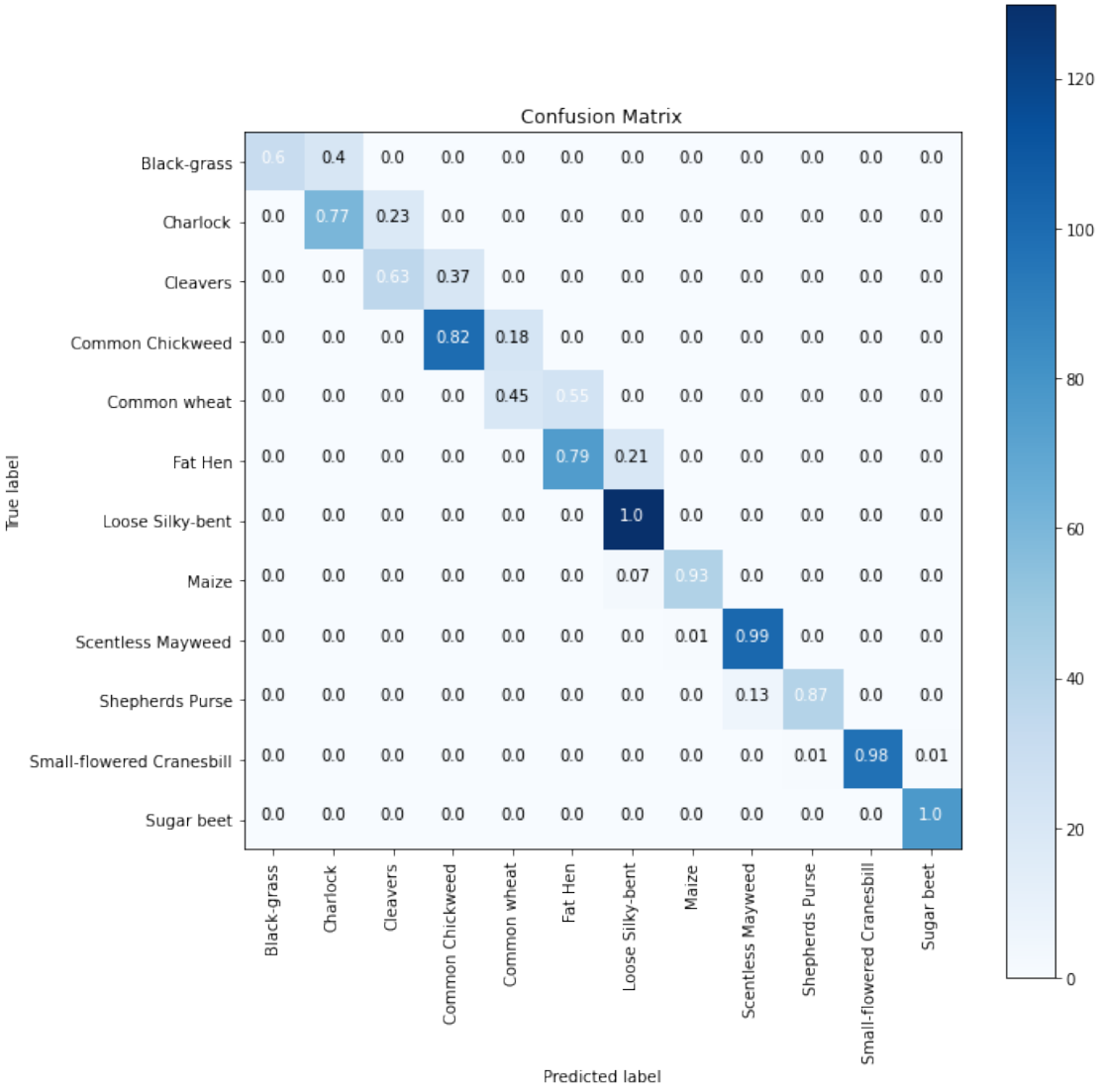



```
y_pred3 = inc_model.predict(validation_generator)
y_pred3 = np.argmax(y_pred3, axis=1)
y_pred3.sort()

cm3 = confusion_matrix(validation_generator.classes, y_pred3)

plot_confusion_matrix(cm3, classes, title='Confusion Matrix')
```

Normalized confusion matrix




```
print(classification_report(validation_generator.classes,
y_pred3,target_names = classes))
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Black-grass | 1.00 | 0.60 | 0.75 | 52 |
| Charlock | 0.74 | 0.77 | 0.75 | 78 |
| Cleavers | 0.67 | 0.63 | 0.65 | 57 |
| Common Chickweed | 0.83 | 0.82 | 0.82 | 122 |
| Common wheat | 0.48 | 0.45 | 0.47 | 44 |
| Fat Hen | 0.76 | 0.79 | 0.77 | 95 |
| Loose Silky-bent | 0.85 | 1.00 | 0.92 | 130 |
| Maize | 0.98 | 0.93 | 0.95 | 44 |
| Scentless Mayweed | 0.94 | 0.99 | 0.97 | 103 |
| Shepherds Purse | 0.98 | 0.87 | 0.92 | 46 |
| Small-flowered Cranesbill | 1.00 | 0.98 | 0.99 | 99 |
| Sugar beet | 0.99 | 1.00 | 0.99 | 77 |

```
              accuracy                           0.85       947
             macro avg       0.85      0.82      0.83       947
          weighted avg       0.86      0.85      0.85       947
```

```
#inc_model.save('inc.h5')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
import tensorflow as tf
import numpy as np
import os
import pandas as pd
from keras.applications.vgg16 import VGG16,preprocess_input


from keras.preprocessing import image

test_dir = '/content/drive/MyDrive/test'

test = []
for file in os.listdir(test_dir):
    test.append(['test/{}'.format(file), file])
test = pd.DataFrame(test, columns=['filepath', 'file'])

test.shape
```

```
(794, 2)
```

```
test.head(2)
```

```
             filepath           file
0  test/003d61042.png  003d61042.png
1  test/02cfeb38d.png  02cfeb38d.png
```

```
INPUT_SIZE = 224

def read_img(filepath, size):
    img = image.load_img(filepath, target_size=size)
    img = image.img_to_array(img)
    return img


x_test = np.zeros((len(test), 224,224, 3), dtype='float32')
i = 0
for file in test['file']:
    img = read_img(os.path.join(test_dir,file), (INPUT_SIZE,
INPUT_SIZE))
    x = preprocess_input(np.expand_dims(img.copy(), axis=0))
    x_test[i] = x
    i+=1
print('test Images shape: {} size: {:,}'.format(x_test.shape,
x_test.size))
```

```
test Images shape: (794, 224, 224, 3) size: 119,519,232
```

```
class_labels = ['Black-grass', 'Charlock', 'Cleavers', 'Common
Chickweed', 'Common wheat', 'Fat Hen', 'Loose Silky-bent',
              'Maize', 'Scentless Mayweed', 'Shepherds Purse', 'Small-
flowered Cranesbill', 'Sugar beet']
```

## VGG

```
vgg = tf.keras.models.load_model('/content/drive/MyDrive/vgg.h5')

vggpred = vgg.predict(x_test)
print('Test bottleneck features shape: {} size:
{:,}'.format(vggpred.shape, vggpred.size))
```

```
Test bottleneck features shape: (794, 12) size: 9,528
```

```
vggpred = np.argmax(vggpred,axis=1)

for i in range(20):
  print(class_labels[vggpred[i]])
```

```
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Common Chickweed
Maize
Common Chickweed
```

## ResNet

```
resnet =
tf.keras.models.load_model('/content/drive/MyDrive/resnet.h5')

resnetpred = resnet.predict(x_test)
print('Test bottleneck features shape: {} size:
{:,}'.format(resnetpred.shape, resnetpred.size))
```

```
Test bottleneck features shape: (794, 12) size: 9,528
```

```
resnetpred = np.argmax(resnetpred,axis=1)


for i in range(20):
  print(class_labels[resnetpred[i]])
```

```
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
Loose Silky-bent
```

## Inception

```
inc = tf.keras.models.load_model('/content/drive/MyDrive/inc.h5')

incpred = inc.predict(x_test)
print('Test bottleneck features shape: {} size:
{:,}'.format(incpred.shape, incpred.size))


Test bottleneck features shape: (794, 12) size: 9,528

incpred = np.argmax(incpred,axis=1)

for i in range(20):
  print(class_labels[incpred[i]])
```

```
Charlock
Charlock
Charlock
Charlock
Charlock
Charlock
Charlock
Small-flowered Cranesbill
Charlock
```

Charlock
Charlock
Small-flowered Cranesbill
Charlock
Charlock
Charlock
Charlock
Charlock
Charlock
Charlock
Charlock